

Beneath the Skull of AI: How It Actually Works (Part 1)

By: Oussama Bounouch



The Great AI Shakeout

- Market shifts are moving from tools to **automated infrastructure**.
- Prompt-engineering hobbies are hitting a wall.
- Survival Strategy: Move from typing prompts to **orchestrating systems**.



Let's get started

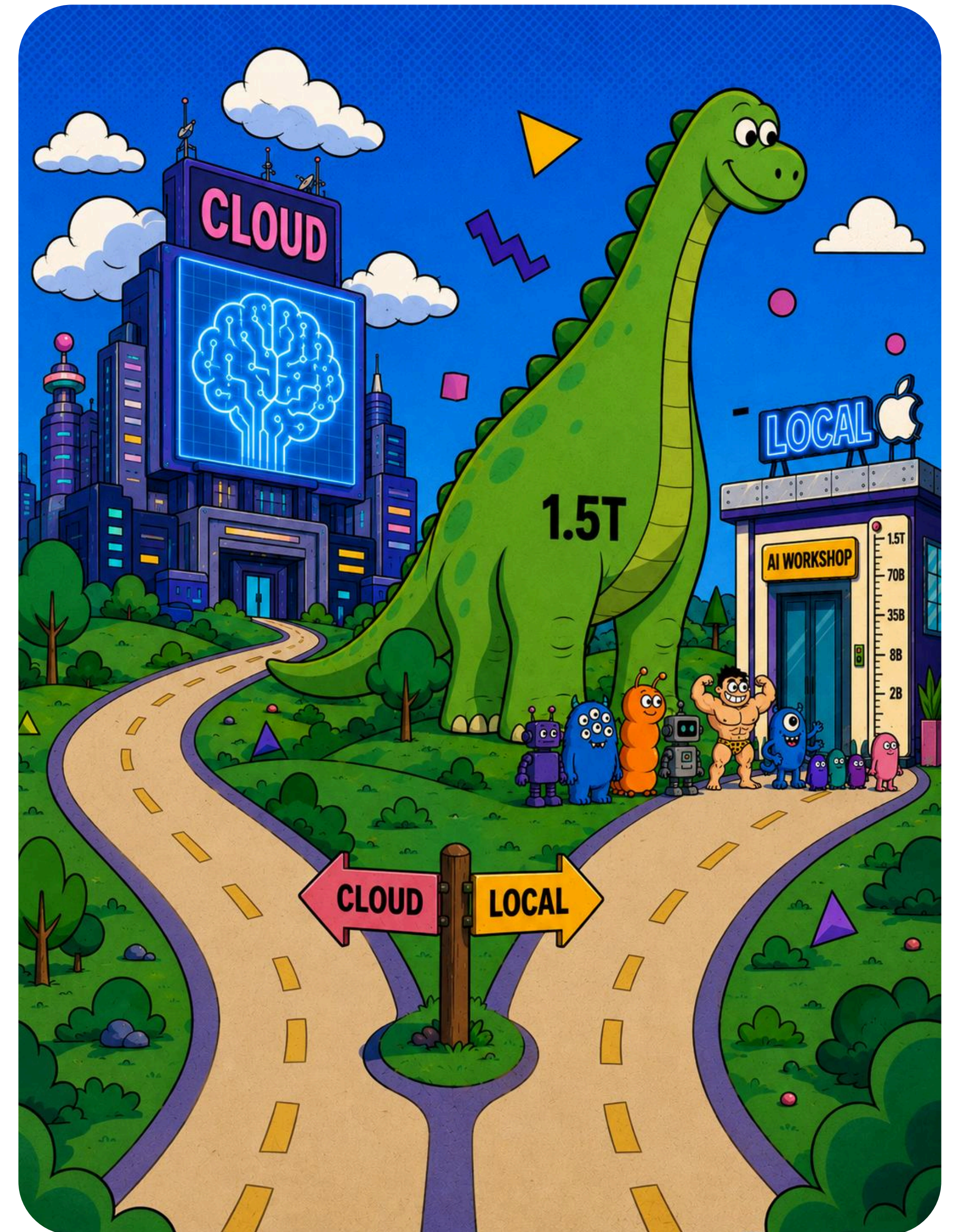
The Crossroads

- **Path 1: Cloud AI**

Use AI models hosted by providers like OpenAI, Anthropic, or Google. The computation happens in their data centers.

- **Path 2: Local AI**

Download and run open-weight models on your own hardware. Your computer performs all the computation.



The Model (LLM)

We are going Right

Choosing your model

- An LLM is a machine trained to predict the next piece of text.
- Model scales: **2B, 8B, 32B, 70B, up to 1T+.**
- More parameters generally allow a model to represent more complex patterns, though they don't automatically make it "**smarter.**"
- The Advantage: **100% Free, 100% Private,** and works completely **offline.**
- The Constraint: You are strictly bound by the **physical limits** of your computer.



Large models

GLM-5.2	~744B (MoE)	Zhipu AI	Advanced agentic workflows, multi-step reasoning, and complex coding automation.
DeepSeek V4	1.6T (MoE)	DeepSeek	Massive context memory, exceptional mathematical reasoning, and highly optimized for complex logic.
Kimi K2.6	~1T+ (MoE)	Moonshot AI	Superior long-context document processing, visual-to-code generation, and multi-agent coordination.
Qwen3.5-397B-A17B	397B Total (17B Active)	Alibaba Cloud	Highly efficient MoE architecture offering flagship-level multilingual and coding performance with lower active compute requirements.
Mistral Large 3	675B	Mistral AI	Enterprise-grade reasoning, top-tier multilingual output, and highly consistent instruction following.

Medium models

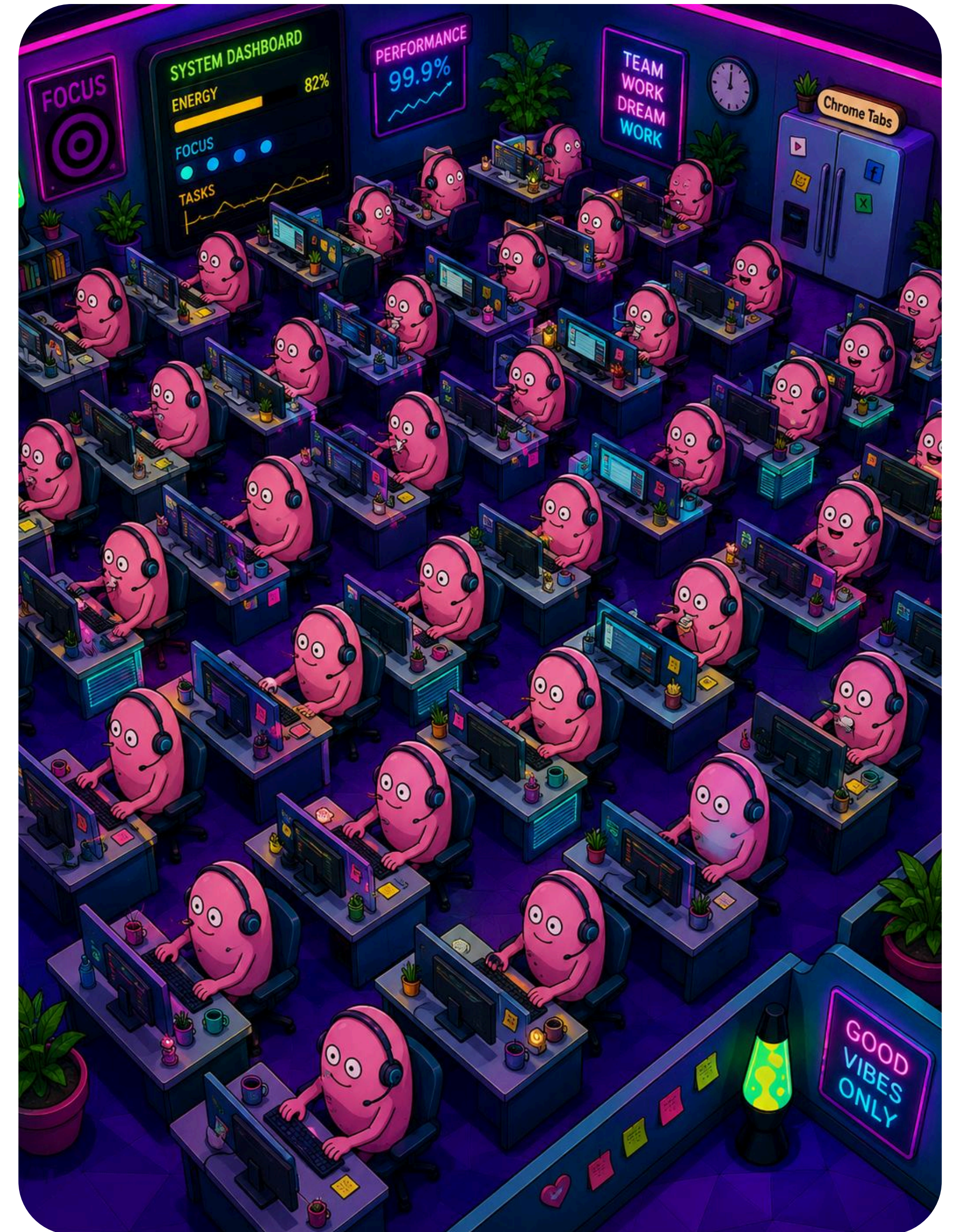
Gemma4-31b	31B	Google DeepMind	Powerful dense model optimized for local consumer hardware, packing strong general reasoning into a manageable footprint.
Gemma4-26b-a4b	26B Total (4B Active)	Google DeepMind	Ultra-efficient MoE designed for high-speed local inference, maximizing token generation speed on standard memory configurations.
Qwen3.6-27b	27B	Alibaba Group	Next-generation dense architecture providing a massive punch for local development, front-end coding, and rapid text generation.
Qwen3.6-35b-a3b	35B Total (3B Active)	Alibaba Group	Extremely lightweight active parameter footprint, perfect for continuous, high-speed background agent execution on personal machines.

Small models

Qwen3.5-9B	9B	Alibaba Group	Best overall balance, strong reasoning & multilingual (100+ languages), excellent coding, dual thinking modes, great agentic performance
Gemma 4 12B QAT	12B	Google DeepMind	Excellent efficiency via QAT (runs ~7GB Q4), strong multimodal (vision + audio), close to 26B performance, great for reasoning & coding on laptops
Granite 4.1 8B	8B	IBM	Top-tier coding & tool calling, strong enterprise RAG performance, high HumanEval score
Gemma 4 E4B	~4.5B (MoE)	Google DeepMind	Best edge/on-device multimodal (text/audio), very low memory, strong agentic/tool use
Llama 3.1 8B Instruct	8B	Meta	Best ecosystem & fine-tunes, solid all-rounder, reliable general chat & instruction following

Mixture of Experts (MoE)

- **The Team:** Imagine a company with hundreds of **specialists**, but only the few who are relevant to your problem join the meeting.
- **The Selection:** An internal "**router**" quickly chooses which experts should answer your request.
- **The Result:** Higher quality answers with less computation, making large AI models faster and cheaper to run.
- Eg: Qwen3.6-35B-**A3B**, Gemma-4-26B-**A4B**



Estimated Sizes of Popular AI Models

Model	Size	Active params	Company
 Claude Sonnet	600–800B	100–200B	Anthropic
 Claude Opus 4.8	1.0–1.5T	200–300B	Anthropic
 Claude Fable 5	2–4T	300–600B	Anthropic
 GPT-5.5	1.5–2.5T	180–250B	OpenAI
 Gemini 3.5 Flash	300–900B	10–16B	Google

The Engine

The Engine (The seat)

- An AI model file is just a giant, frozen list of raw **decimal numbers** (weights).
- The Engine is the software runner that reads those numbers and **executes** the math.
- Without an **engine**, the model is just dead weight sitting in your basement storage closet.





The Structural Savior llama.cpp

- The Core Open-Source Breakthrough
- Created by Georgi Gerganov to bypass massive corporate software dependencies.
- Written in pure, lightweight C/C++ to run directly on consumer hardware.
- The silent foundation supporting almost all local AI software today.



Ollama – The Friendly Front-End

- Simplifying Local Deployment
- Wraps the raw power of `llama.cpp` into clean, automated background processes.
- Handles model downloads, hardware detection, and API endpoints with single commands.



Hardware Prerequisites

RAM / vRAM / GPU / Bandwidth

RAM vs vRAM

- **System RAM:** Huge capacity but slower. It manages your basic computer tasks, like running your operating system, keeping browser tabs open, and multitasking.
- **GPU vRAM:** Ultra-fast but smaller. It lives right inside the graphics chip and does the heavy lifting required to keep the AI responding and typing at top speed.
- **The Bottleneck:** If a model's size exceeds vRAM, it overflows into system RAM, causing generation speeds to slow dramatically.



Unified Memory Architecture (UMA) (The exception)

- Traditional PCs **split** memory between System RAM and GPU VRAM.
- Apple Silicon uses a single **Unified Memory pool** shared by the CPU and GPU.
- This lets large AI models use much more memory without needing enterprise GPUs.



The RAM Allocation Reality Check

- Your AI model never gets the **entire memory pool**.
- The Carpet (macOS): Permanently occupies **~4–6 GB**.
- The Double-Door Fridge (Chrome, Slack, etc.): Quickly takes another **~2–4 GB**.



The weight of the raw Brains

- Raw models (**FP16**) require **~2GB** of RAM for every **1 Billion** parameters.
- An **unoptimized 35B** model demands a massive **70GB** of free space.
- Result: The system chokes, and performance collapses.



The GPU Muscle

- **vRAM** determines if a model can physically fit inside your room.
- **GPU's Bandwidth:** The active muscle that determines how fast the model can think.



The Bandwidth

- The doorway that determines how **fast** information can actually be fed to those muscles.
- Why it matters: Even with a huge room (VRAM) and powerful muscles (GPU), a narrow doorway (memory bandwidth) leaves the GPU **waiting for data instead of working**, slowing AI generation.



The AI Speed Formula

In the world of local AI development, engineers use a technique called quantization to find the sweet spot between **Speed & Precision**.

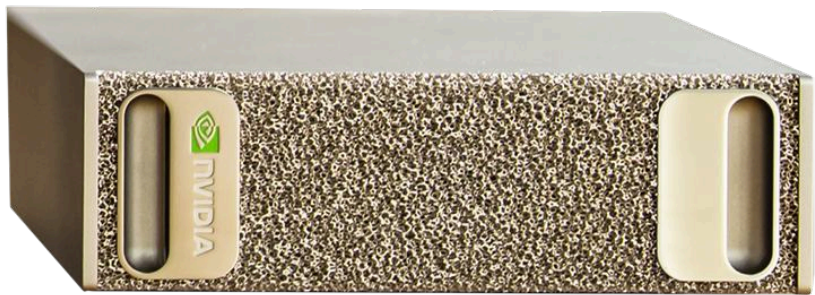
$$\text{Speed (Token/s)} = \frac{\text{Memory bandwidth (GB/s)}}{\text{Model size (GB)}}$$



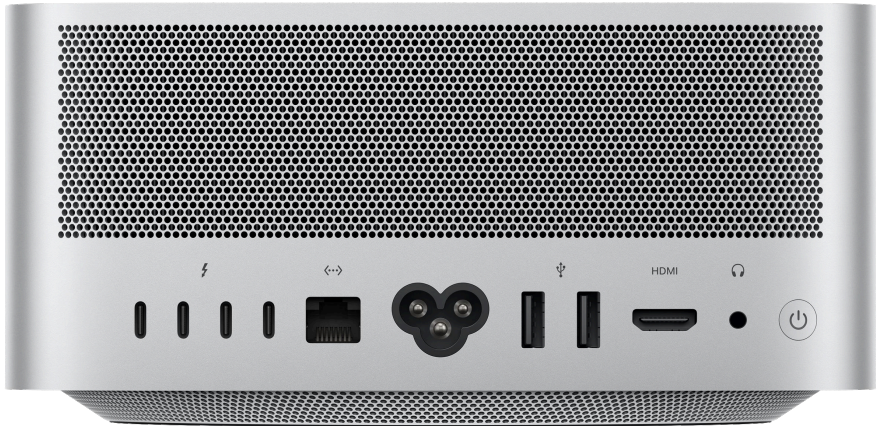
Tokens/s running Qwen 3.6 35B A3B (70GB)



Mac Mini M4



NVIDIA DGX Spark



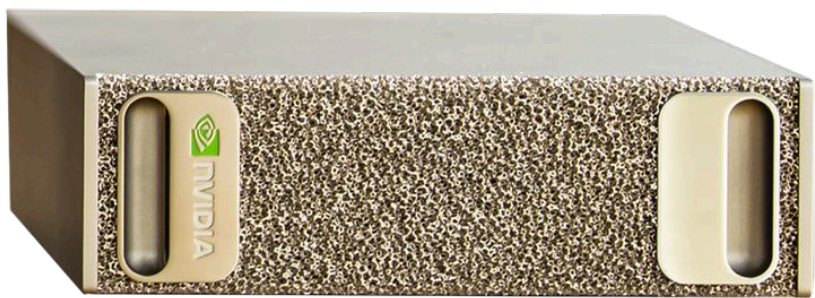
Mac Studio M3 Ultra

Bandwidth	120 GB/s	273 GB/s	819 GB/s
Tokens /s	??	??	??

Tokens/s running Qwen 3.6 35B A3B (70GB)



Mac Mini M4



NVIDIA DGX Spark



Mac Studio M3 Ultra

Bandwidth

120 GB/s

273 GB/s

819 GB/s

Tokens /s

~1.7 tokens

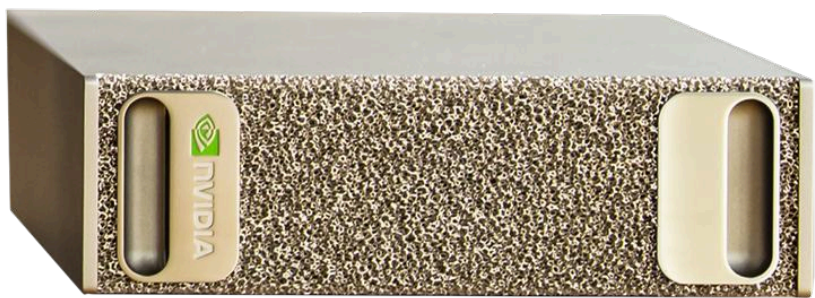
~3.9 tokens

~11.7 tokens

Tokens/s running Qwen 3.6 35B A3B (70GB)



Mac Mini M4



NVIDIA DGX Spark



Mac Studio M3 Ultra

Bandwidth

120 GB/s

273 GB/s

819 GB/s

Tokens /s

~20 tokens

~46 tokens

~137 tokens

The Quantization

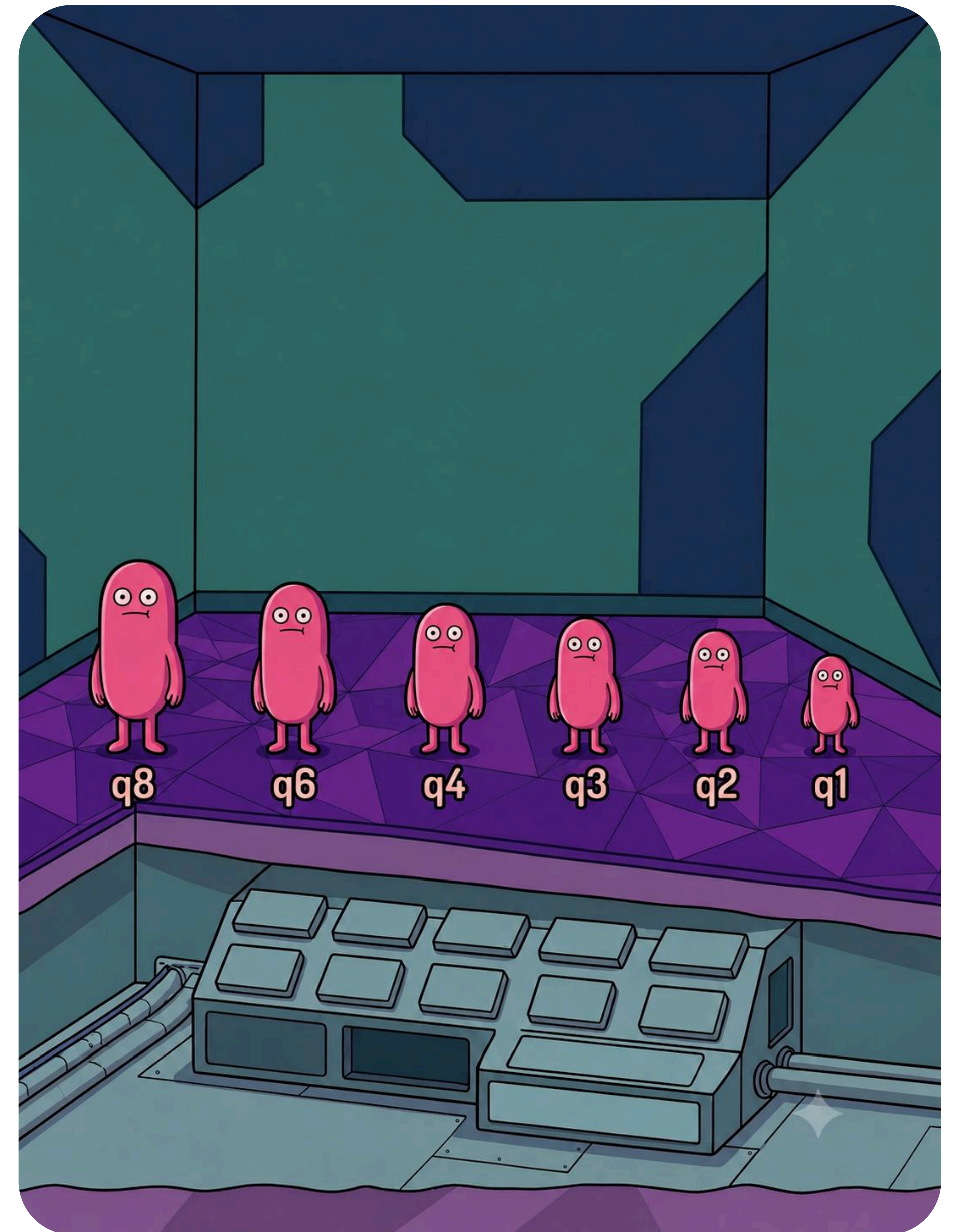
The Mathematical Shrink Ray

- **Quantization:** The process of compressing a model's file weight by reducing its **numerical precision**.
- Allows giant, heavy minds to comfortably **fit** onto consumer-grade **hardware**.



How Precision Scaling Works

- **Raw Precision (FP16):** Uses highly complex 16-bit floating decimals for every single link.
- **Q8 / Q6:** **High precision**, heavy footprint, minor compression.
- **Q5 / Q4:** **The Optimal Sweet Spot**, cuts file size by ~70-75% with negligible accuracy drops.
- **Q2 / Q1:** **The Destruction Zone**, cripples the model's cognitive capacity.



Qwen3.6-35B-A3B

Quantized versions

53 GB of vRAM saved

Hardware compatibility [Log In](#) to add your hardware

1-bit			UD-IQ1_M 10 GB
2-bit	UD-IQ2_XXS 10.8 GB	UD-IQ2_M 11.5 GB	UD-Q2_K_XL 12.3 GB
3-bit	UD-IQ3_XXS 13.2 GB	UD-IQ3_S 13.7 GB	UD-Q3_K_S 15.4 GB
		UD-Q3_K_M 16.6 GB	UD-Q3_K_XL 16.8 GB
4-bit	UD-IQ4_XS 17.7 GB	UD-Q4_K_S 20.9 GB	MXFP4_MOE 21.7 GB
	UD-IQ4_NL 18 GB	UD-Q4_K_M 22.1 GB	UD-IQ4_NL_XL 19.5 GB
			UD-Q4_K_XL 22.4 GB
5-bit	UD-Q5_K_S 24.9 GB	UD-Q5_K_M 26.5 GB	UD-Q5_K_XL 26.6 GB
6-bit		UD-Q6_K 29.3 GB	UD-Q6_K_XL 31.8 GB
8-bit		Q8_0 36.9 GB	UD-Q8_K_XL 38.5 GB
16-bit			BF16 69.4 GB

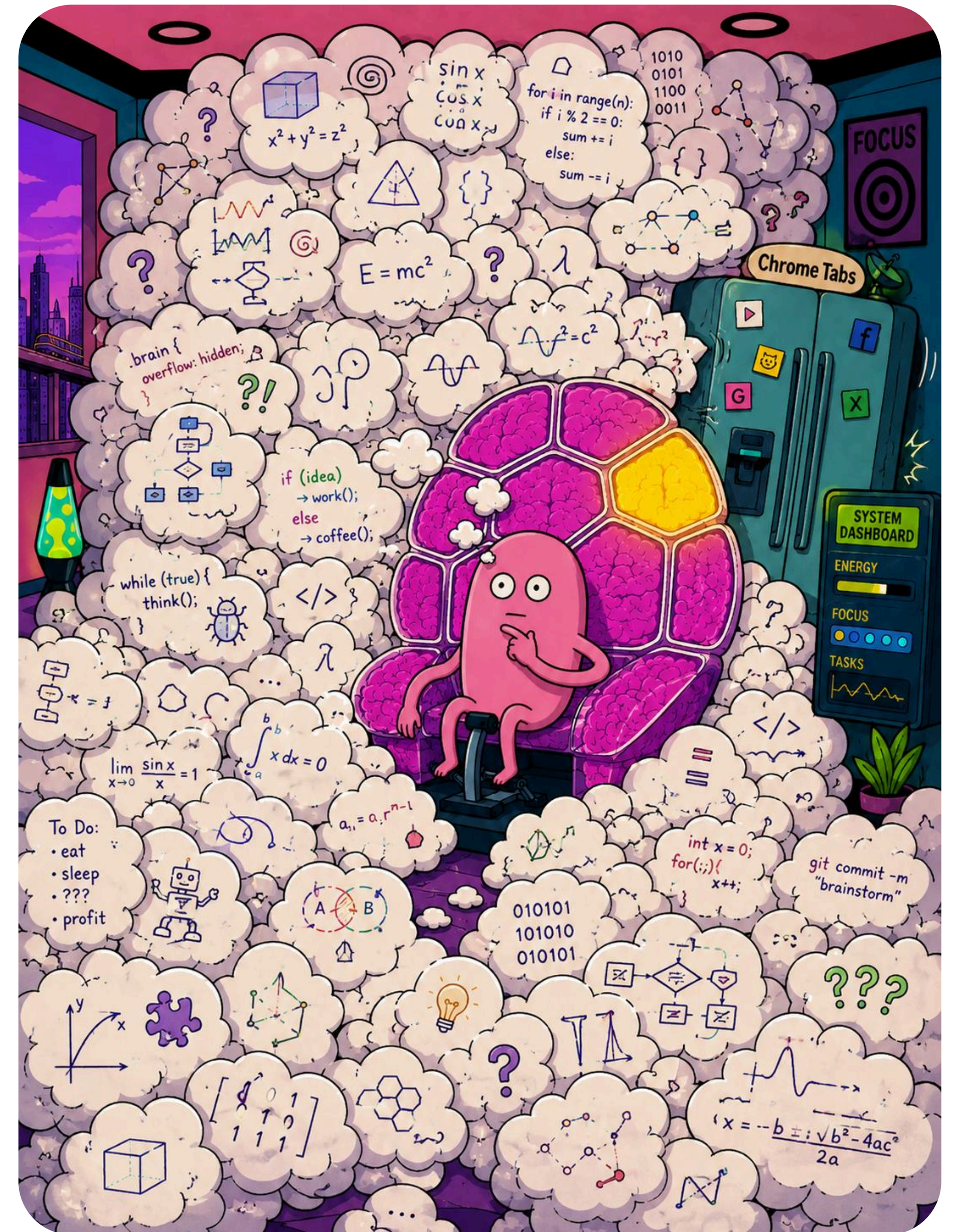
Estimated precision loss : Qwen3.6-35B-A3B

Quantization	Size	Estimated Precision Retained	Estimated Quality Loss	Recommendation
BF16	~70 GB	100%	0%	Reference model
Q8_0	36.9 GB	99.8%	0.2%	Virtually indistinguishable from BF16
Q6_K	29.3 GB	99.2%	0.8%	Excellent if RAM allows
Q5_K_M	26.5 GB	98.5–99%	1–1.5%	Best balance of quality and size
Q4_K_M	22.1 GB	96.5–97.5%	2.5–3.5%	★ Sweet spot for most users
Q3_K_M	16.6 GB	91–93%	7–9%	Usable, reasoning noticeably weaker
IQ2_M	11.5 GB	80–84%	16–20%	Significant degradation
IQ1_M	10.0 GB	65–75%	25–35%	Mostly experimental

The Context

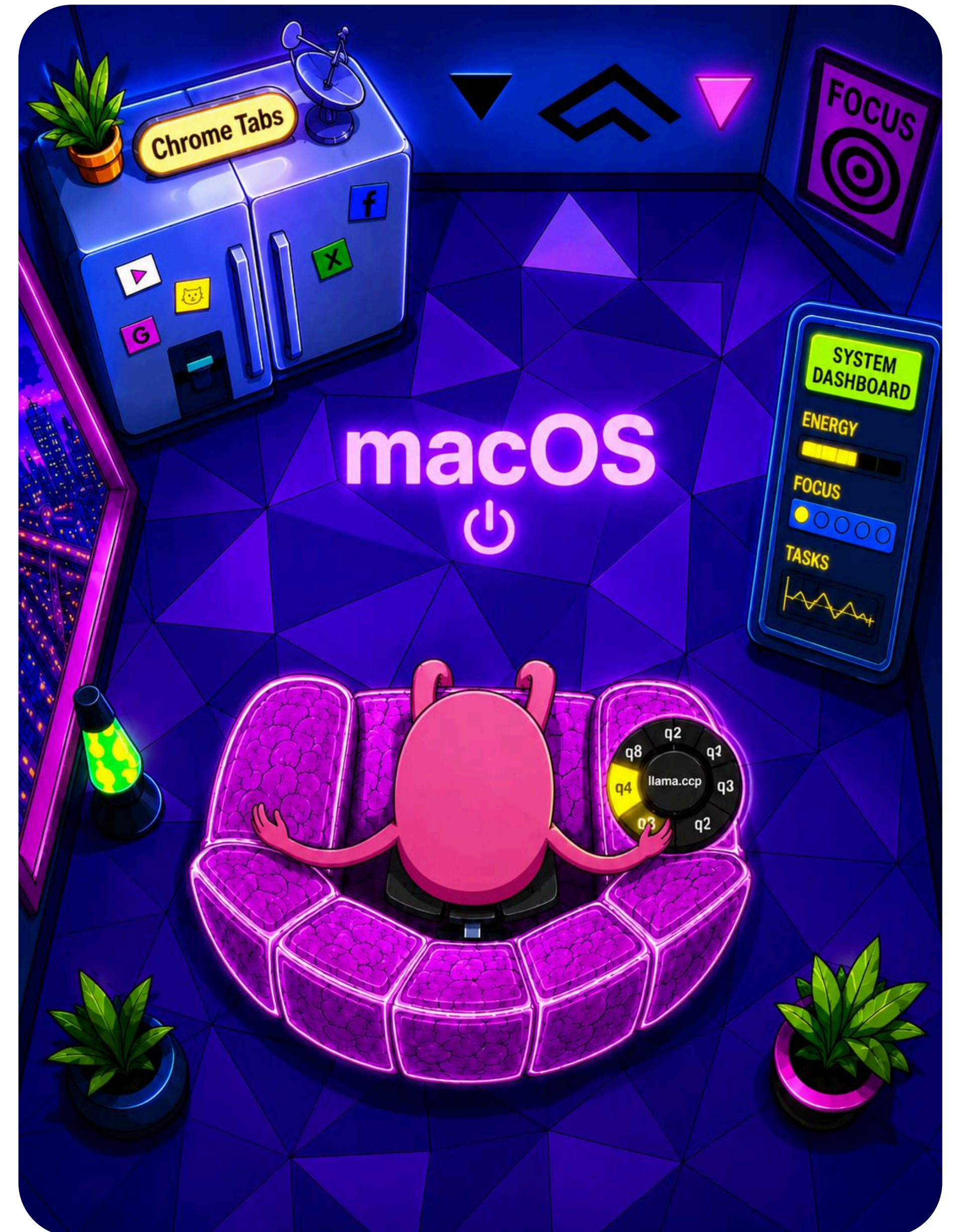
Understanding Conversation Overhead

- The AI keeps a mathematical notebook of everything you've already said. Engineers call this **the KV Cache**.
- To avoid re-reading the whole conversation from scratch, the engine caches past meanings.
- **The Catch:** The longer the text string grows, the more physical RAM this cloud devours.



Choosing the right context size

- **Hardware Limit:** Set the maximum context length based on your available RAM. Larger contexts use more memory.
- **Safety Limit:** Keeping a reasonable maximum prevents memory spikes that can slow down or crash your system.
- **200k → 120k → 80k → 32k → 4k**



Context Compression

- **The Review:** The AI reads the conversation and identifies the most important information.
- **The Cleanup:** It removes repeated details and less important messages to reduce memory usage.
- **The Summary:** A short summary is kept, allowing the conversation to continue while using much less memory.



Context Quantization

- **The Compression:** The AI stores its conversation memory in a more compact format, reducing RAM usage.
- **The Benefit:** Less memory per conversation means you can handle much longer context windows.
- **The Trade-off:** Lower precision may slightly reduce accuracy, but the memory savings are often worth it.



Today, we understood how the brain works...

But a brain alone can't accomplish much.

In Part 2, we'll give it:

👁 Eyes to see images and documents.

🖐 Hands to interact with software and the web.

🔧 Tools to search, calculate, write code, and automate tasks.

🧠 Memory to remember goals and previous work.

⚡ Autonomy to plan and complete tasks on its own.

